Chapter 5: Evaluating Predictive Performance

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate.

Import required packages

Make sure DMBA package is available

In [1]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

In [9]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import mlba
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, auc, roc_curve
from sklearn.model_selection import train_test_split
```

To illustrate the use of predictive accuracy measures and charts of prediction error distribution, consider the error metrics and charts shown in Table 5.1 and Figure 5.1. These are the result of fitting a certain predictive model to prices of used Toyota Corolla cars. The training set includes 600 cars and the validation set includes 400 cars. Results are displayed separately for the training and validation sets. We can see from the histogram and boxplot corresponding to the validation set that most errors are in the [-2000, 2000] range.

Table 5.1

Load file and generate training and validation sets.

In [4]:
```
#VB67
# Load data frame and reduce it to 1000 rows for regression analysis
car_df = mlba.load_data('ToyotaCorolla.csv')
car_df = car_df.iloc[0:1000]

# create a list of predictor variables by removing output variables and text
excludeColumns = ('Price', 'Id', 'Model', 'Fuel_Type', 'Color')
predictors = [column for column in car_df.columns if column not in excludeCo
outcome = 'Price'

# partition data
X = car_df[predictors]
y = car_df[outcome]
train_X, valid_X, train_y, valid_y = train_test_split(X, y, test_size=0.4, r

# train linear regression model
reg = LinearRegression()
reg.fit(train_X, train_y)

# evaluate performance
# training
mlba.regressionSummary(y_true=train_y, y_pred=reg.predict(train_X))
# validation
mlba.regressionSummary(y_true=valid_y, y_pred=reg.predict(valid_X))
```

```
Regression statistics

Mean Error (ME) : -0.0000
Root Mean Squared Error (RMSE) : 1133.0721
Mean Absolute Error (MAE) : 838.3022
Mean Percentage Error (MPE) : -0.8205
Mean Absolute Percentage Error (MAPE) : 7.4933

Regression statistics

Mean Error (ME) : 133.2043
Root Mean Squared Error (RMSE) : 1131.0105
Mean Absolute Error (MAE) : 876.5051
Mean Percentage Error (MPE) : 0.2694
Mean Absolute Percentage Error (MAPE) : 7.8791
```

Figure 5.1

In [10]:
```
#VB67
#calculate training residual values
pred_error_train = pd.DataFrame({
    'residual': train_y - reg.predict(train_X),
    'data set': 'training'
})
#calculate validation residual values
pred_error_valid = pd.DataFrame({
```

```
    'residual': valid_y - reg.predict(valid_X),
    'data set': 'validation'
})
boxdata_df = pd.concat([pred_error_train, pred_error_valid])
#plot the graphs
#HISTOGRAMS AND BOXPLOTS OF TOYOTA PRICE PREDICTION ERRORS, FOR TRAINING AND
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(8, 4))
common = {'bins': 100, 'range': [-6500, 6500]}
pred_error_train.hist(ax=axes[0], **common)
pred_error_valid.hist(ax=axes[1], **common)
boxdata_df.boxplot(ax=axes[2], by='data set')

axes[0].set_title('training')
axes[1].set_title('validation')
axes[2].set_title(' ')
axes[2].set_ylim(-6500, 6500)
plt.suptitle('Prediction errors')
plt.subplots_adjust(bottom=0.15, top=0.85, wspace=0.35)

plt.show()
```

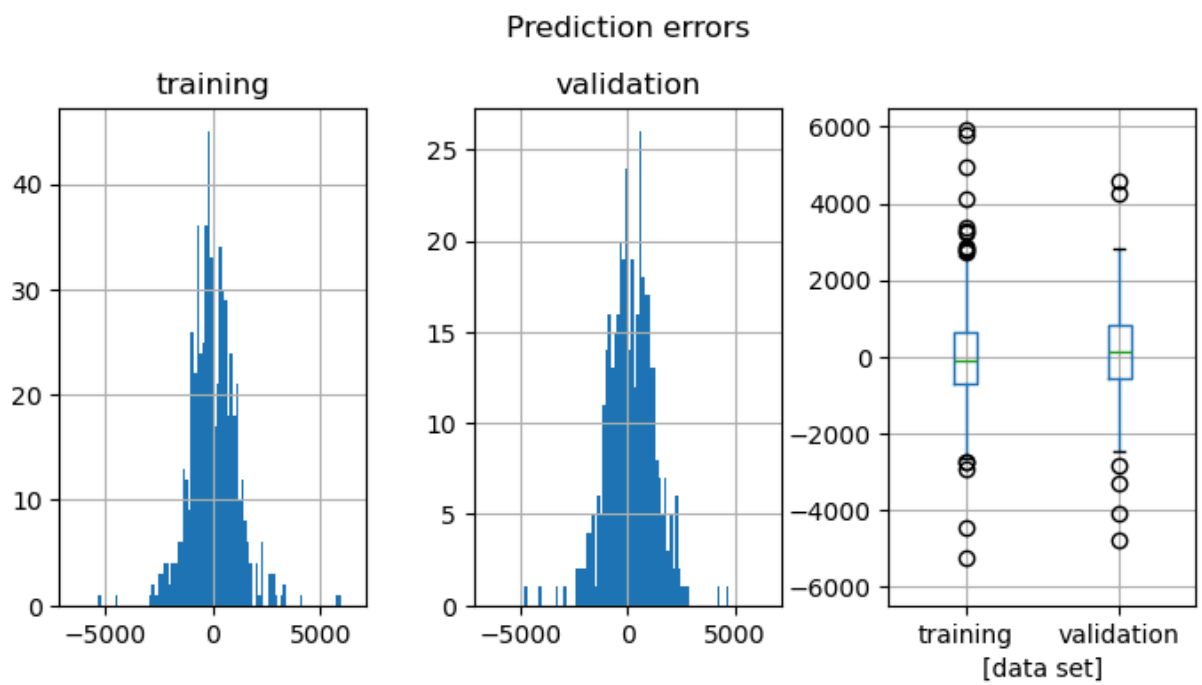
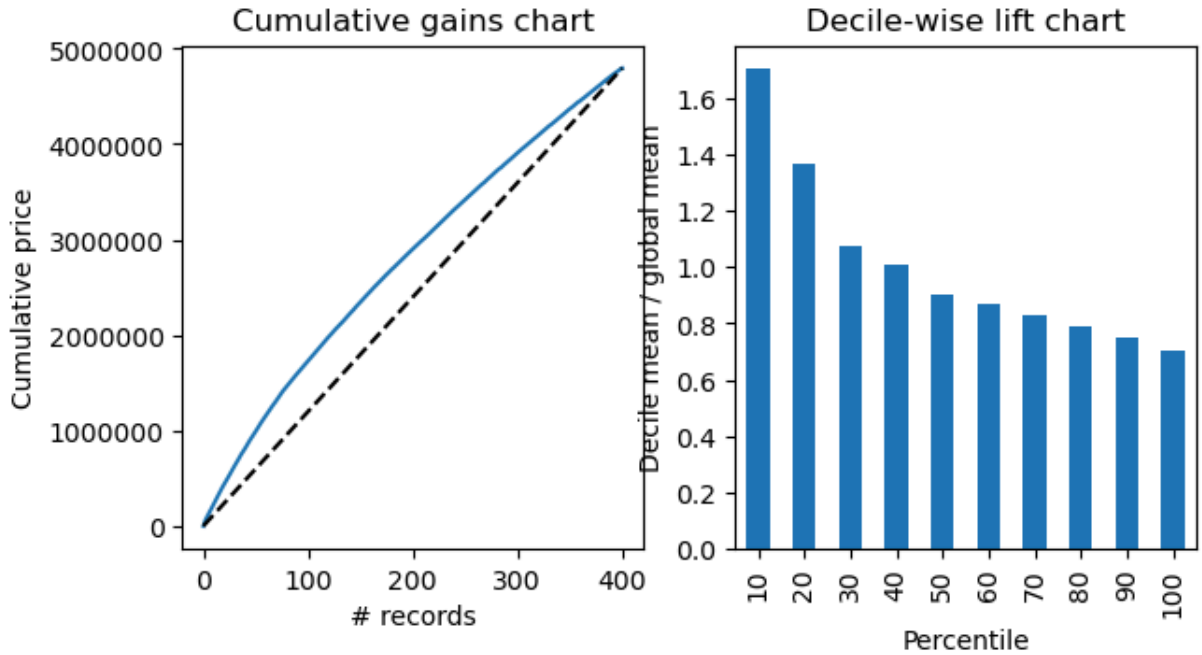


Figure 5.2

In [40]:
```
#VB67
# sort the actual values in descending order of the prediction
df = pd.DataFrame({'predicted': reg.predict(valid_X),
                   'actual': valid_y})
#plot the graphs
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(7, 3.5))
ax = mlba.gainsChart(df, ranking='predicted', actual='actual', type='regress
                     ax=axes[0])
ax.set_ylabel('Cumulative price')
ax.ticklabel_format(style='plain')
```

```
mlba.liftChart(df, ranking='predicted', actual='actual', ax=axes[1], labelBa
plt.show()
```



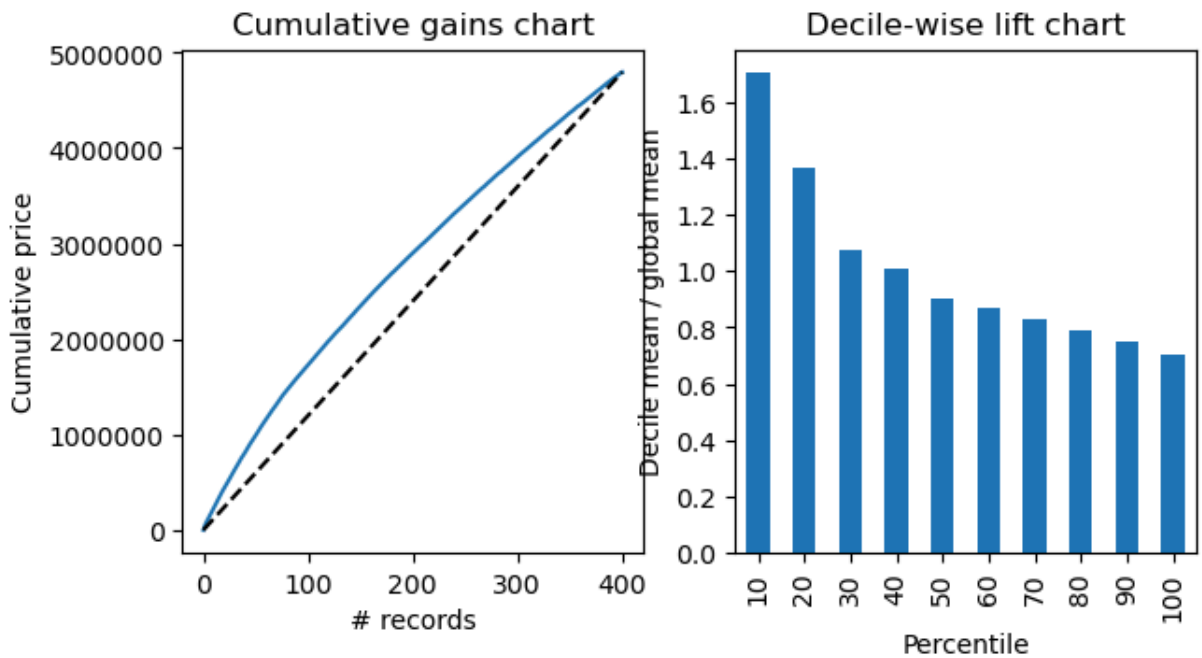


Table 5.5

function mlba.classificationSummary is included in the mlba package

In [20]:
```
#VB67
#loading dataframe and displaying the first 5 rows
owner_df = mlba.load_data('ownerExample.csv')
owner_df.head()
#class_names = ['nonowner', 'owner']
```

Out[20]:

	Class	Probability
0	owner	0.9959
1	owner	0.9875
2	owner	0.9844
3	owner	0.9804
4	owner	0.9481

In [18]:
```
#VB67
# cutoff = 0.5
#predicting class as owner/nonowner based on the cutoff value
predicted = ['owner' if p > 0.5 else 'nonowner' for p in owner_df.Probabilit
mlba.classificationSummary(y_true=owner_df.Class, y_pred=predicted)
```

```
Confusion Matrix (Accuracy 0.8750)

         Prediction
  Actual nonowner    owner
nonowner       10        2
   owner        1       11
```

In [15]:
```
#VB67
# cutoff = 0.25
#predicting class as owner/nonowner based on the cutoff value
predicted = ['owner' if p > 0.25 else 'nonowner' for p in owner_df.Probabili
mlba.classificationSummary(y_true=owner_df.Class, y_pred=predicted)
```

```
Confusion Matrix (Accuracy 0.7917)

         Prediction
  Actual nonowner    owner
nonowner        8        4
   owner        1       11
```

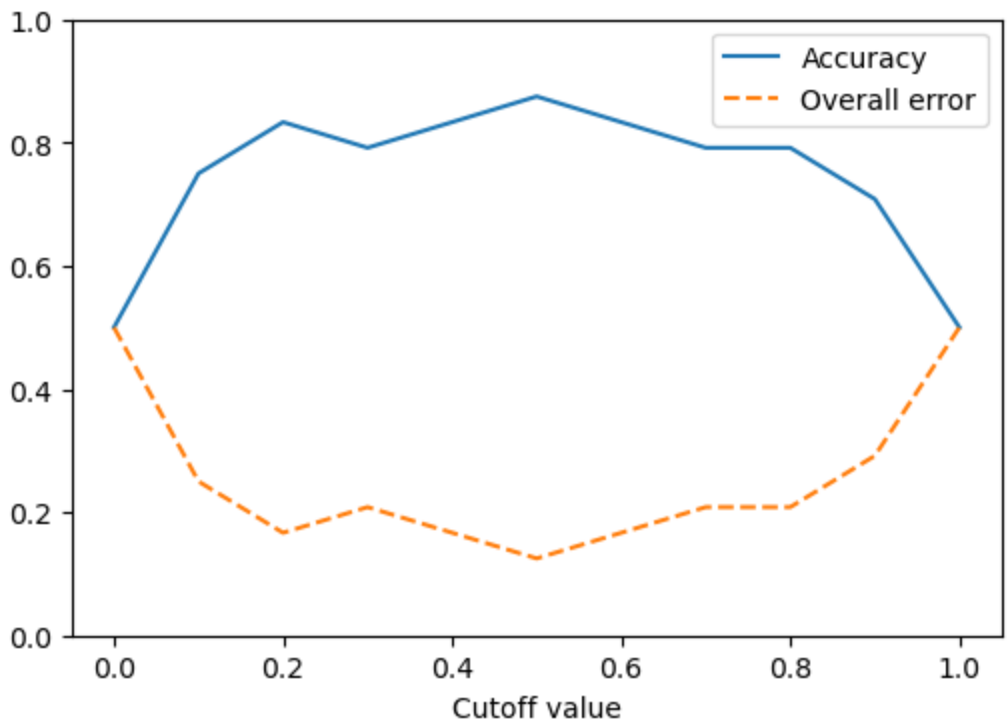
In [16]:
```
#VB67
# cutoff = 0.75
#predicting class as owner/nonowner based on the cutoff value
predicted = ['owner' if p > 0.75 else 'nonowner' for p in owner_df.Probabili
mlba.classificationSummary(y_true=owner_df.Class, y_pred=predicted)
```

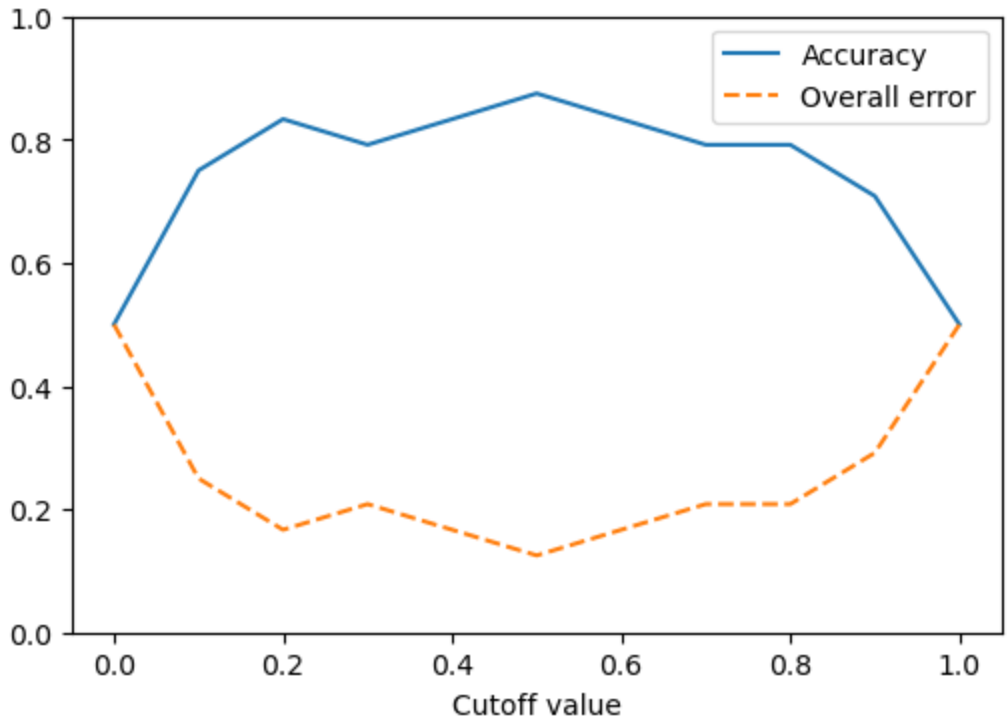
```
Confusion Matrix (Accuracy 0.7500)

         Prediction
  Actual nonowner    owner
nonowner       11        1
   owner        5        7
```

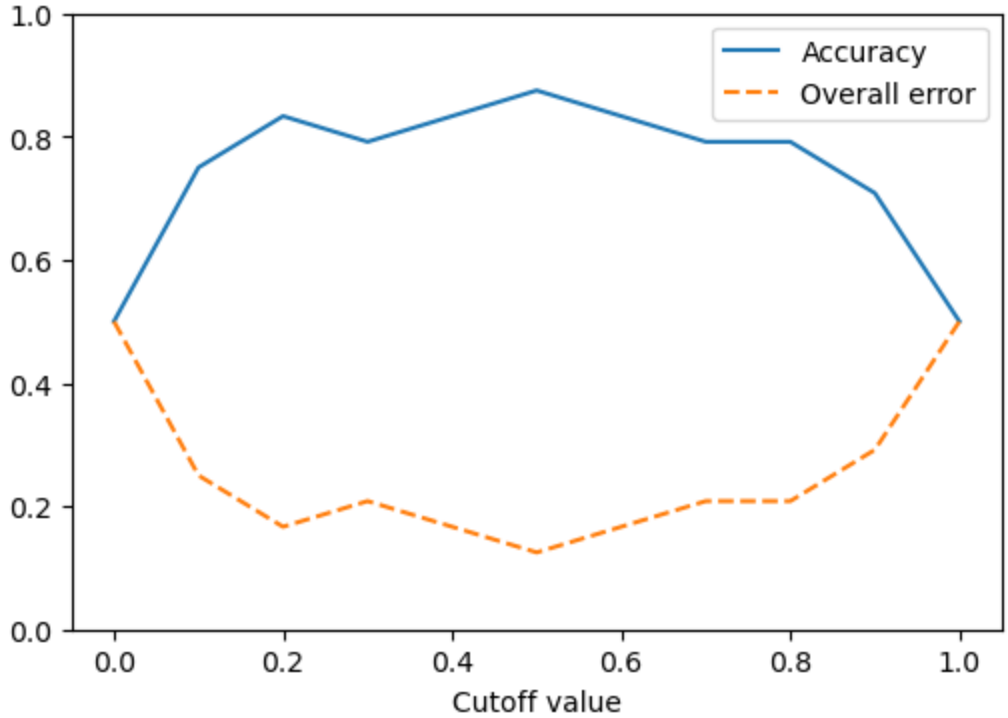
Figure 5.4

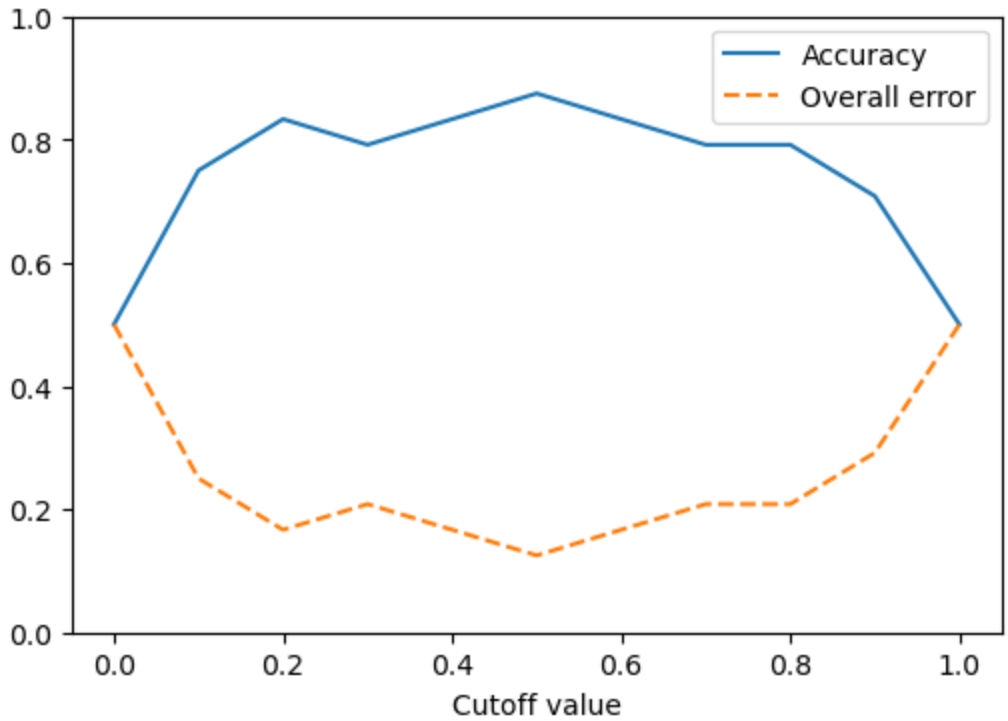
Calculate the accuracy at different cutoff values and create a graph.

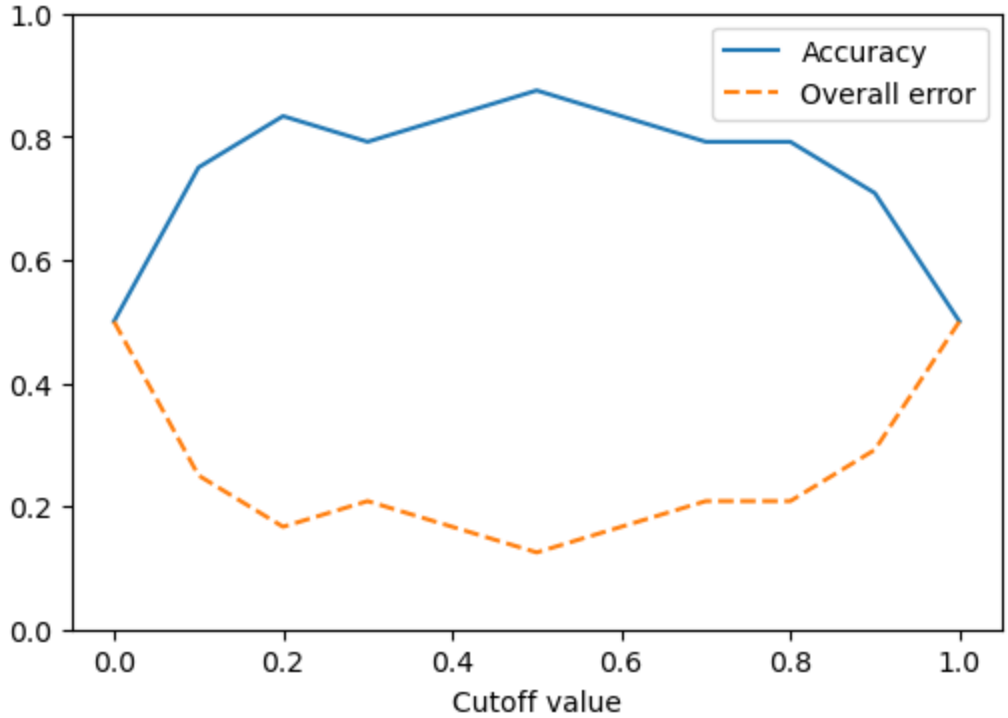
In [35]:
```
#VB67
#loading dataframe
df = mlba.load_data('liftExample.csv')
cutoffs = [i * 0.1 for i in range(11)]
accT = []
for cutoff in cutoffs:
    predicted = [1 if p > cutoff else 0 for p in df.prob]
    accT.append(accuracy_score(df.actual, predicted))
# PLOTTING ACCURACY AND OVERALL ERROR AS A FUNCTION OF THE CUTOFF VALUE (RID
fig, ax = plt.subplots()
fig.set_size_inches(6, 4)
ax.plot(cutoffs, accT, '-', label='Accuracy')
ax.plot(cutoffs, [1 - acc for acc in accT], '--', label='Overall error')
ax.set_ylim(0, 1)
ax.set_xlabel('Cutoff value')
ax.legend();
plt.show()
```

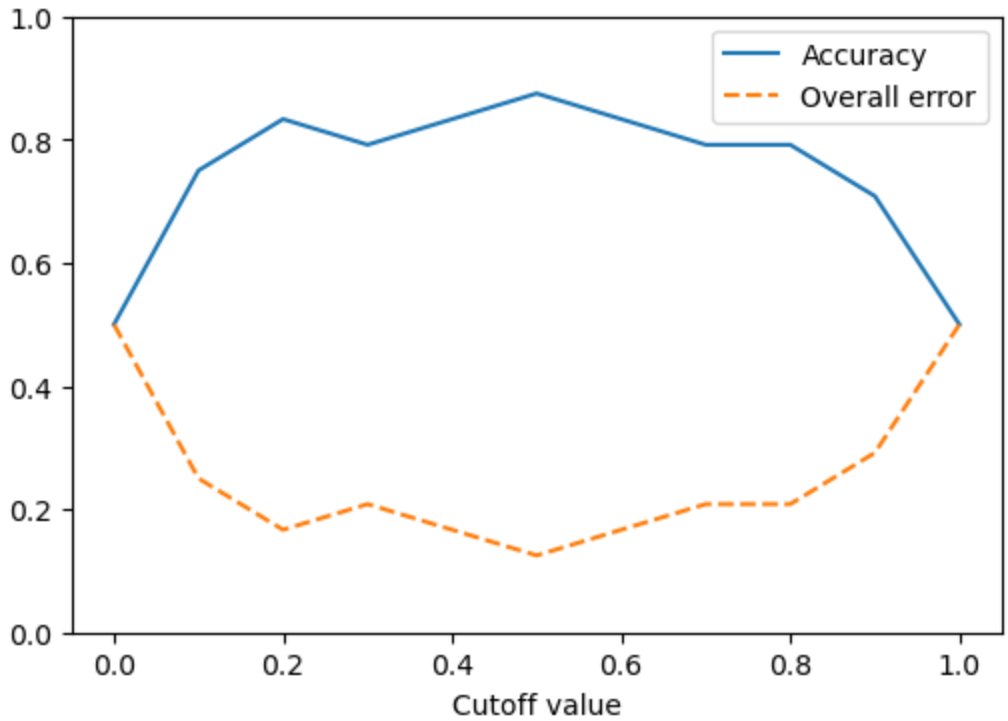


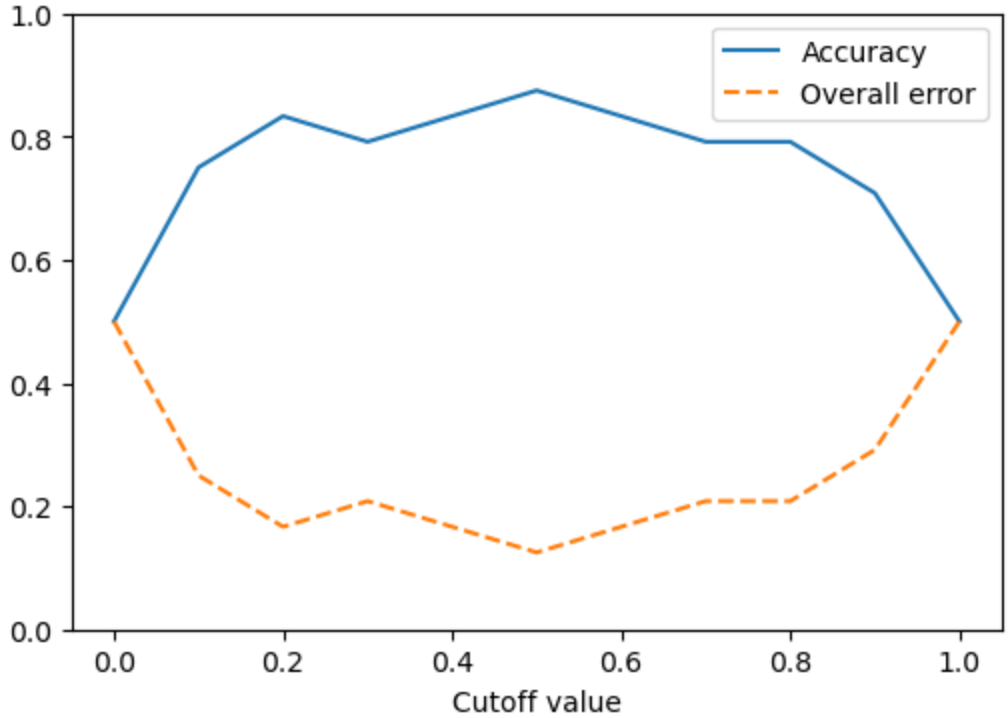

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

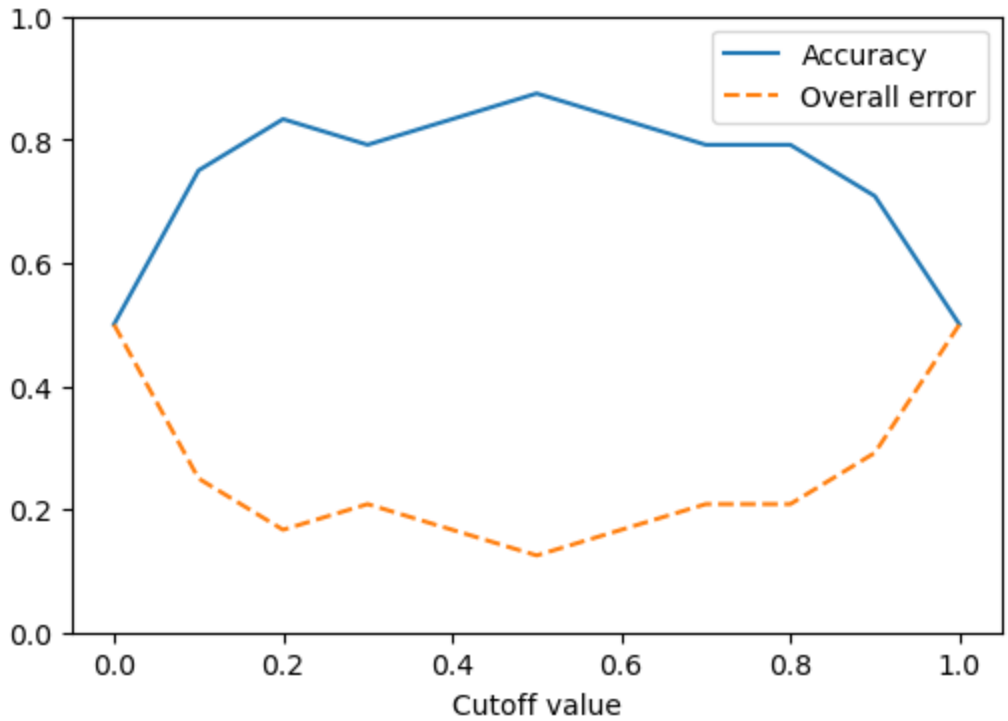

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

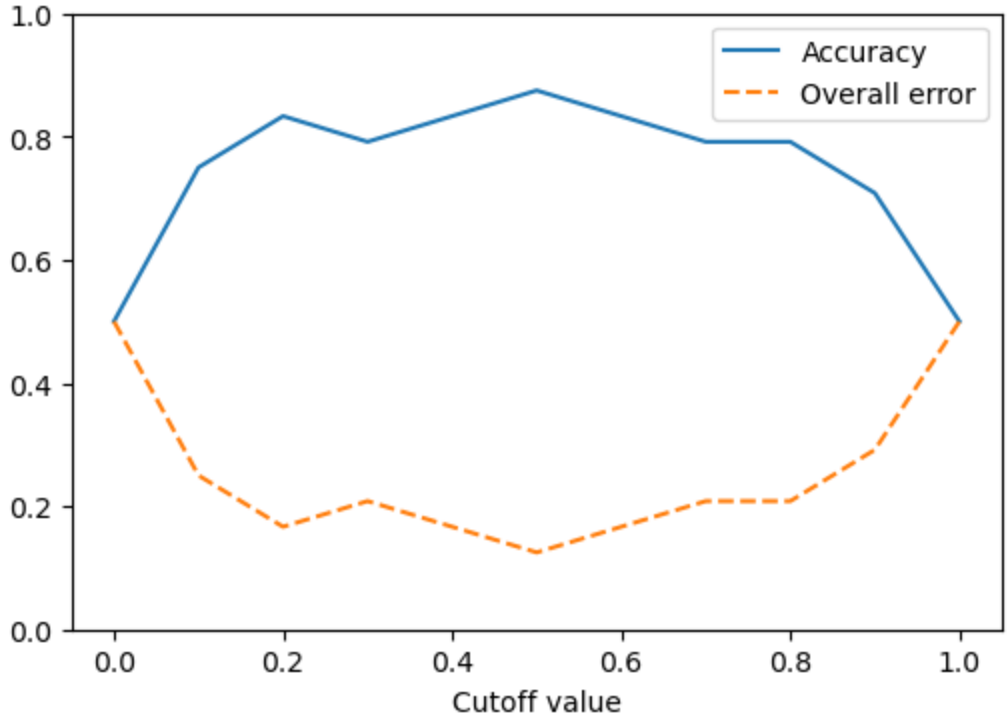

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

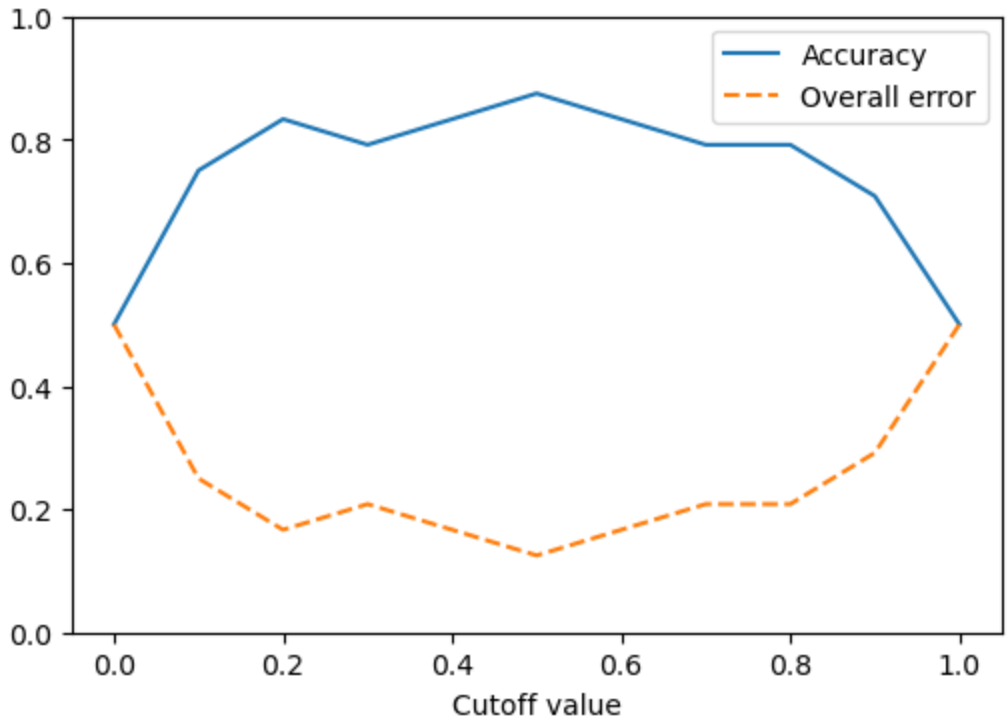

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

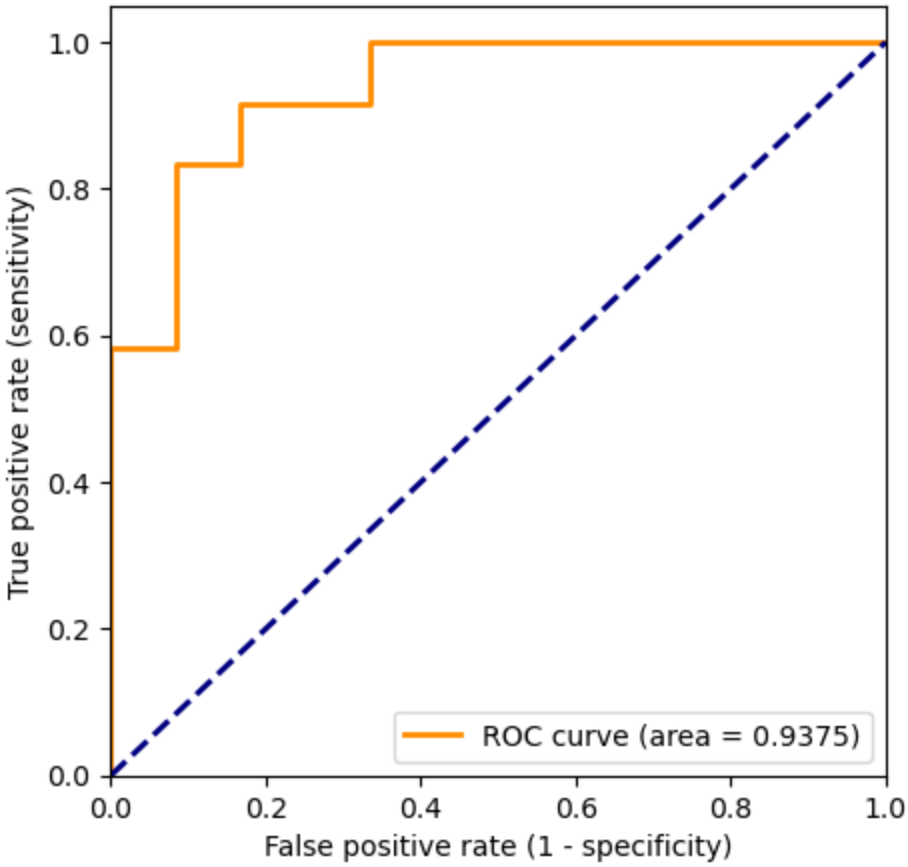

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

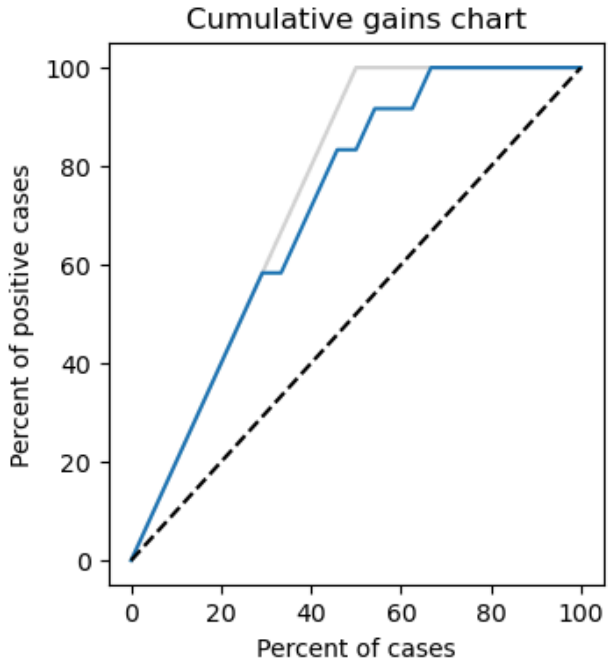

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

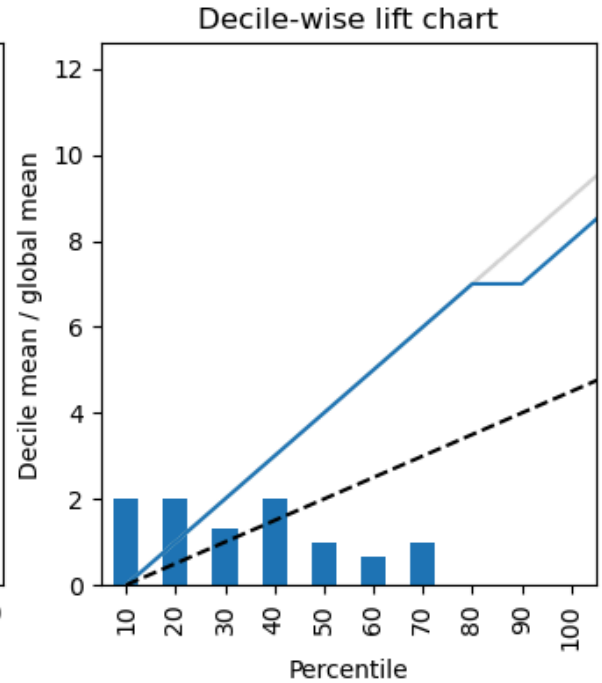

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error

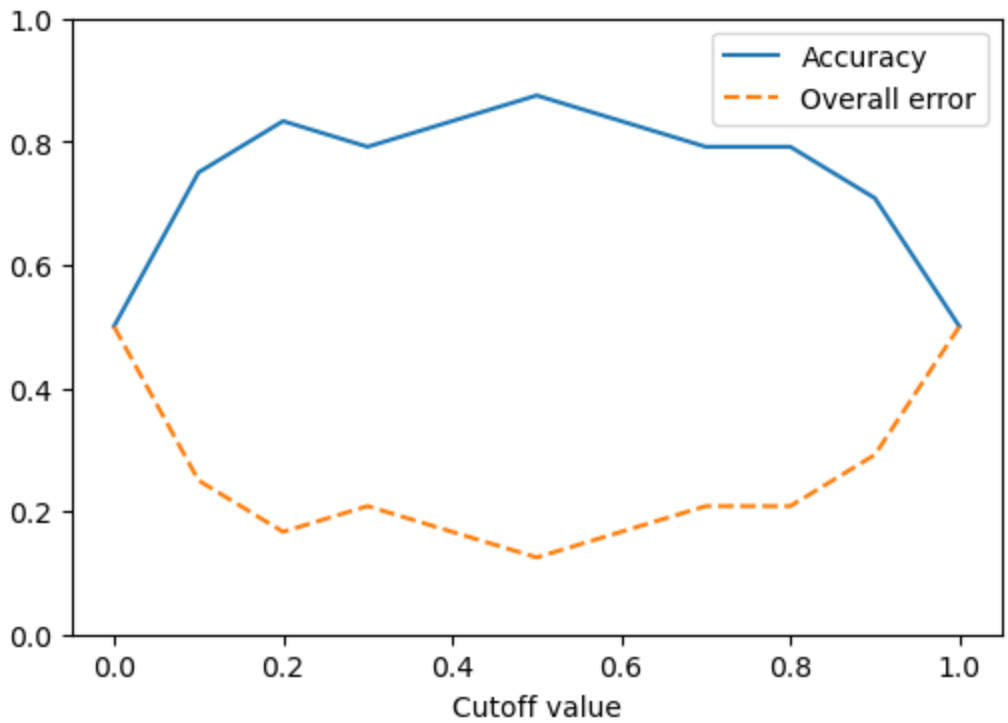

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value
Accuracy
Overall error


Accuracy
Overall error
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Cutoff value


ROC curve (area = 0.9375)
True positive rate (sensitivity)
False positive rate (1 - specificity)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


Cumulative gains chart
Percent of positive cases
Percent of cases
0
20
40
60
80
100


Decile-wise lift chart
Decile mean / global mean
Percentile
0
2
4
6
8
10
12
10
20
30
40
50
60
70
80
90
100


Accuracy
Overall error
Cutoff value
0.0
0.2
0.4
0.6
0.8
1.0

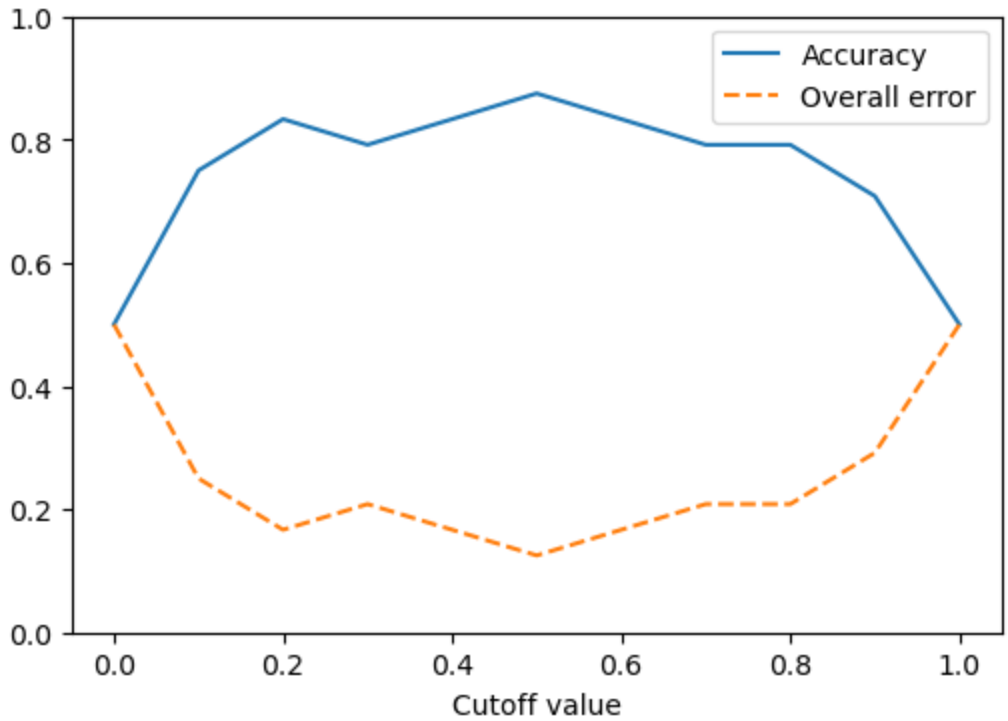


Figure 5.5

In [36]:
```
#VB67
# compute ROC curve and AUC
fpr, tpr, _ = roc_curve(df.actual, df.prob)
roc_auc = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=[5, 5])
ax.plot(fpr, tpr, color='darkorange',
        lw=2, label=f'ROC curve (area = {roc_auc:0.4f})')
ax.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
ax.set_xlim([0.0, 1.0])
ax.set_ylim([0, 1.05])
ax.set_xlabel('False positive rate (1 - specificity)')
ax.set_ylabel('True positive rate (sensitivity)')
ax.legend(loc="lower right");
plt.show()
```

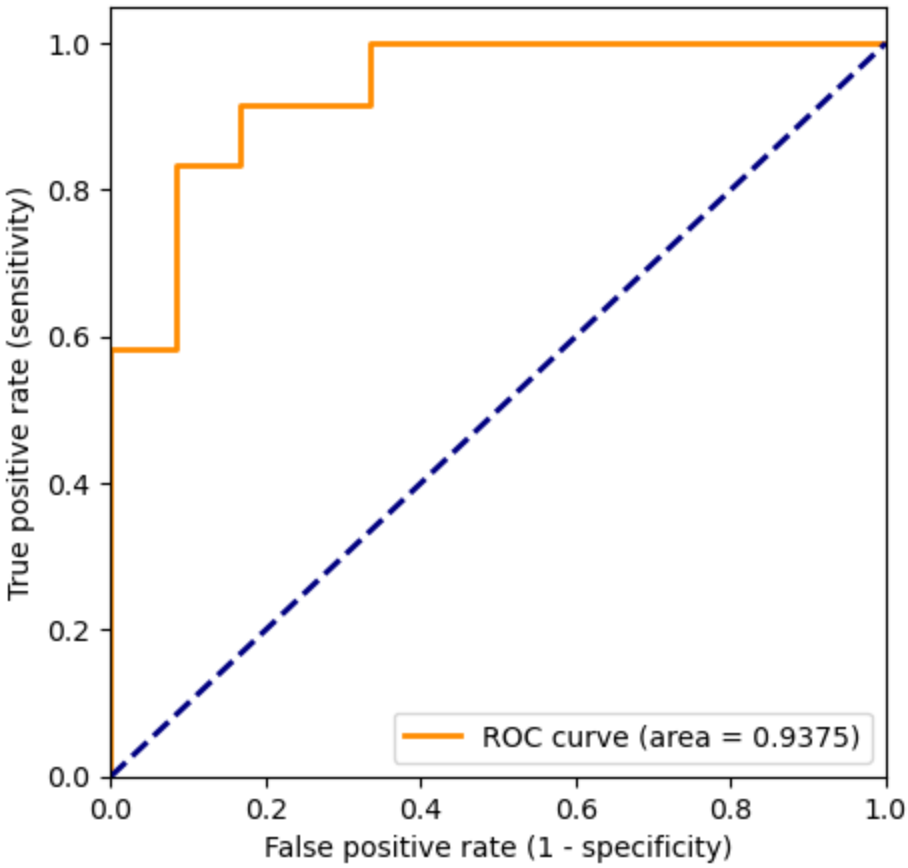


Figure 5.6

In [37]:
```
#VB67
#loading dataframe and plotting CUMULATIVE GAINS CHART FOR THE MOWER EXAMPLE
df = mlba.load_data('liftExample.csv')
fig, axes = plt.subplots(ncols=2, figsize=(8, 4))
mlba.gainsChart(df, ranking='prob', actual='actual', ax=axes[0])
mlba.gainsChart(df, ranking='prob', actual='actual', show_counts=True, ax=ax
plt.show()
```

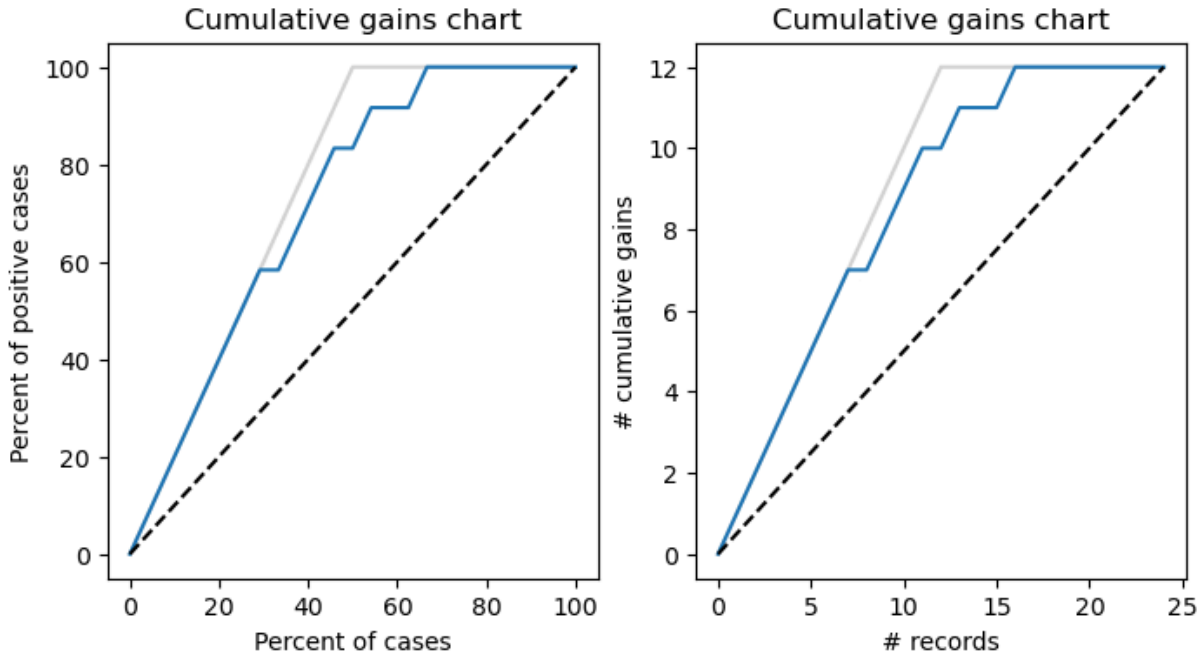
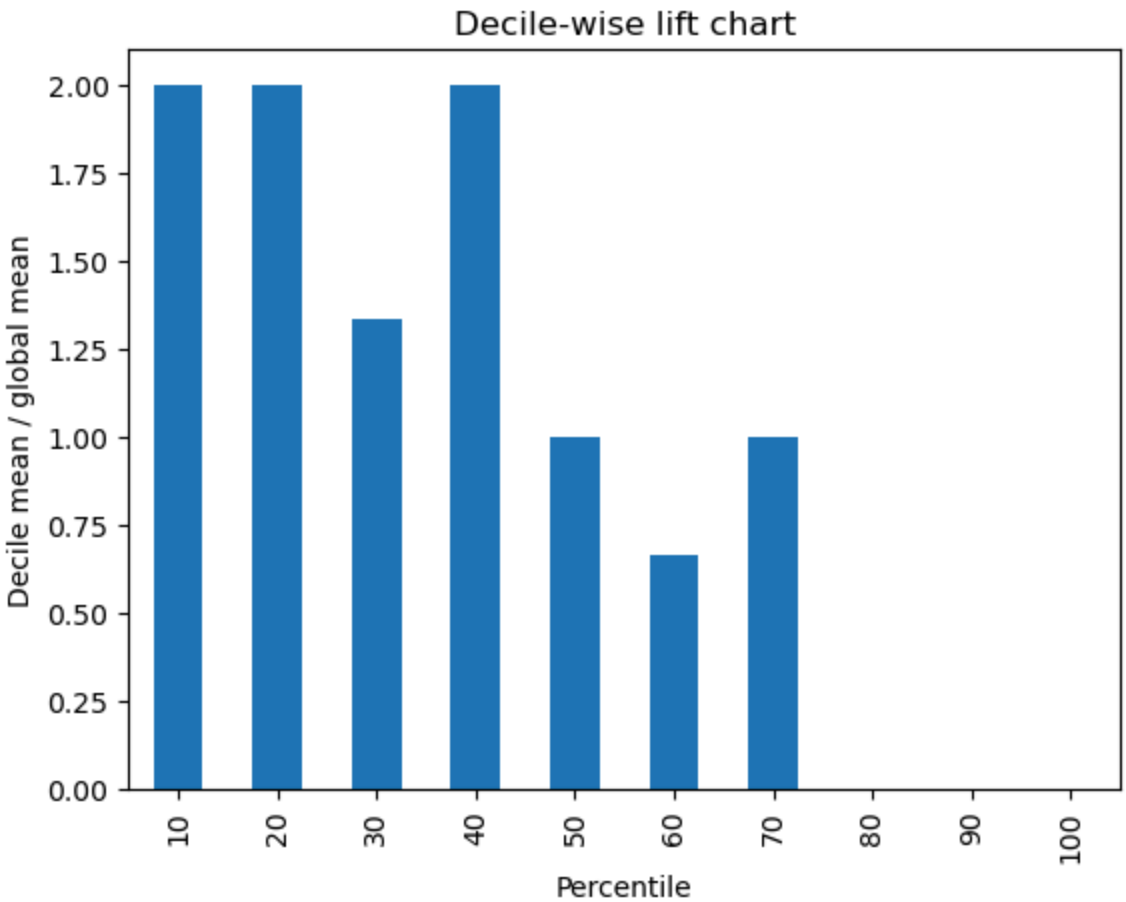


Figure 5.7

In [38]:
```
#VB67
# DECILE-WISE LIFT CHART
mlba.liftChart(df, ranking='prob', actual='actual', labelBars=False);
plt.show()
```


Decile-wise lift chart
Decile mean / global mean
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
10
20
30
40
50
60
70
80
90
100
Percentile

In [ ]: